HEALTHCARE APPOINTMENT QUEUE & PRIORITY MANAGEMENT SYSTEM

A PROJECT REPORT

Submitted by

VARUN OJHA(23BCS14265)

DEEPTANU SAHA(24BCS80037)

AJIT KUMAR RAI(23BCS11916)

DHARMENDRA KUMAR(24BCS80040)

in partial fulfillment for the award of the degree of

BACHELOR OF ENGINEERING

IN

COMPUTER SCIENCE AND ENGINEERING

Chandigarh University

APRIL 2026

BONAFIDE CERTIFICATE

Certified that this project report "**HEALTHCARE APPOINTMENT QUEUE & PRIORITY MANAGEMENT SYSTEM**" is the bonafide work of "**Deeptanu Saha (24BCS80037), VARUN OJHA (23BCS14265), AJIT KUMAR RAI (23BCS11916), DHARMENDRA KUMAR (24BCS80040)**" who carried out the project work under my/our supervision.

SIGNATURE	**SIGNATURE**
DR. SANJEEV KUMAR	MR. RAHUL DURAGAPAL
HEAD OF THE DEPARTMENT Computer Science & Engineering	**PROJECT SUPERVISOR** Computer Science & Engineering

INTERNAL EXAMINER	**EXTERNAL EXAMINOR**

TABLE OF CONTENTS

LIST OF FIGURES i

LIST OF TABLES i

ABSTRACT ii

GRAPHICAL ABSTRACT… iii

LIST OF ABBREAVIATIONS iv

LIST OF SYMBOLS v

CHAPTER 1 – INTRODUCTION 1

1.1 Problem Identification 1

1.2 Objectives 2

1.3 Scope of the Project 2

1.4 Organization of the Report 3

CHAPTER 2 – DESIGN FLOW / PROCESS 4

2.1 System Architecture 4

2.2 Design Constraints & Considerations 5

2.3 Alternative Designs Considered 5

2.4 Best Design Selection 6

2.5 System Flow – Algorithm and Flowchart 6

CHAPTER 3 – RESULTS ANALYSIS AND VALIDATION 8

3.1 Implementation Details 8

3.2 API Endpoints 10

3.3 Database Design 10

3.4 Output Screenshots 11

3.5 Testing and Validation 14

CHAPTER 4 – CONCLUSION AND FUTURE WORK 15

4.1 Conclusion 15

4.2 Future Work 16

REFERENCES 18

APPENDIX – USER MANUAL 19

LIST OF FIGURES

Figure 2.1: System Architecture Diagram 4

Figure 2.2: System Data Flow Diagram 7

Figure 2.3: Priority Queue Algorithm Flowchart 7

Figure 3.1: Patient Intake Form – Registration Panel 11

Figure 3.2: Priority Queue – All Patients View (sorted by severity) 11

Figure 3.3: Priority Queue – CRITICAL Filter (severity 8–10) 12

Figure 3.4: Priority Queue – HIGH Priority Filter (severity 6–7) 12

Figure 3.5: Priority Queue – LOW Priority Filter (severity 1–3) 13

Figure 3.6: Full Dashboard View – Split Panel Layout 13

LIST OF TABLES

Table 2.1: Technologies Used 4

Table 2.2: Priority Classification by Severity Score 6

Table 3.1: REST API Endpoints……10

Table 3.2: Database Schema – patient table 10

Table 3.3: System Test Cases and Results 14

ABSTRACT

The Healthcare Appointment Queue & Priority Management System is a web-based application developed to optimize patient flow and improve service efficiency in hospitals and clinics. Traditional queue systems often follow a first-come, first-served approach, which can delay treatment for critically ill patients. This system addresses that limitation by introducing a priority-based queue mechanism that ensures patients with more severe conditions receive immediate medical attention.

The application allows healthcare staff to register patients by entering key details such as name, department, disease condition, and a severity score ranging from 1 to 10. Based on this score, patients are dynamically organized into a priority queue where higher severity cases are given precedence. In situations where multiple patients share the same severity level, the system ensures fairness by prioritizing patients based on their arrival time using a first-in-first-out (FIFO) approach as a secondary criterion.

The system is built using a full-stack architecture. The frontend is developed using modern JavaScript technologies, providing an interactive and user-friendly interface for patient registration and queue monitoring. The backend is implemented using the Spring Boot framework, which handles business logic, priority queue processing, and RESTful API communication. A MySQL database is used for efficient data storage, retrieval, and management of patient records.

Core functionalities of the system include real-time queue updates, dynamic filtering of patients based on priority levels (Critical, High, Medium, Low), and visual indicators such as color-coded severity levels to enhance usability. Additionally, the system incorporates a live waiting time estimator that calculates approximate delays based on queue length and processing rate. Healthcare staff can also perform actions such as marking patients as attended, rescheduling appointments, or removing entries from the queue.

From a technical perspective, the system demonstrates the practical implementation of data structures such as priority queues, comparator-based sorting algorithms, and efficient database indexing for faster query performance. It also follows RESTful design principles, ensuring scalability, modularity, and ease of integration with other healthcare systems.

Furthermore, the system can be extended with advanced features such as authentication and role-based access control, integration with electronic health records (EHR), notification systems (SMS/Email alerts), and predictive analytics for patient load forecasting. Overall, this project provides a scalable, efficient, and intelligent solution for modern healthcare queue management, significantly improving patient experience and operational workflow.

GRAPHICAL ABSTRACT

The Healthcare Appointment Queue & Priority Management System follows a structured workflow in which hospital staff register patients by entering details such as name, department, disease, and severity score. The frontend interface communicates with a Spring Boot backend through RESTful APIs, where business logic is applied to process patient data. The backend implements a priority queue algorithm that sorts patients based on severity (descending) and arrival time (ascending) to ensure fair and efficient prioritization. All patient records are stored in a MySQL database for persistence. The processed data is then displayed on a real-time dashboard, enabling healthcare staff to monitor the queue, filter patients by priority levels, and take actions such as attending or removing patients. This system ensures that critical patients receive immediate attention while improving overall hospital workflow and reducing waiting time.

Figure : System Flow Diagram

LIST OF ABBREVIATIONS

- **API** Application Programming Interface
- **REST** Representational State Transfer
- **UI** User Interface
- **MVC** Model-View-Controller
- **DB** Database
- **SQL** Structured Query Language
- **JPA** Java Persistence API
- **ORM** Object Relational Mapping
- **HTTP** HyperText Transfer Protocol
- **JSON** JavaScript Object Notation
- **CRUD** Create, Read, Update, Delete
- **OPD** Outpatient Department
- **EHR** Electronic Health Record
- **EMR** Electronic Medical Record
- **FIFO** First In First Out

LIST OF SYMBOLS

- **S** — Severity Score (1–10)
- **T** — Arrival Time (Timestamp)
- **P** — Priority Level
- **Q** — Patient Queue
- → — Data Flow Direction
- ≥ — Greater than or equal to
- ≤ — Less than or equal to
- ∑ — Total / Aggregation
- **Δt** — Waiting Time Difference
- # — Rank Position in Queue

CHAPTER 1

INTRODUCTION

In modern healthcare systems, efficient patient flow management is essential to ensure timely diagnosis and treatment. Hospitals and clinics often face challenges such as overcrowding, long waiting times, and inefficient prioritization of patients, especially during peak hours or emergency situations. Traditional queue management approaches, which are mostly manual or based on simple first-come, first-served policies, are not suitable for environments where patient conditions can vary significantly in severity.

This project presents a Healthcare Appointment Queue & Priority Management System, a software-based solution designed to automate and optimize patient prioritization. The system assigns a severity score to each patient at the time of registration and dynamically organizes the queue so that patients with more critical conditions are attended to first. Critical emergencies such as cardiac arrest, stroke, or severe trauma are given the highest priority, ensuring faster medical response and potentially saving lives.

The system is designed as a real-time, user-friendly platform for hospital staff, including receptionists and doctors. It provides a centralized dashboard where patient information is displayed clearly with color-coded indicators, enabling quick decision-making. By reducing manual intervention and improving transparency, the system enhances both operational efficiency and quality of care.

1.1 Problem Identification

Traditional hospital queuing systems suffer from several limitations that directly impact patient care and hospital efficiency. Most systems rely on manual processes or simple chronological ordering, which fail to consider the urgency of medical conditions. This can lead to critical patients waiting longer than they should, increasing health risks.

The key problems identified are:

- Lack of automated severity-based prioritization during patient registration.
- Inability for healthcare staff to quickly identify high-risk or emergency patients.
- Dependence on manual or paper-based systems, leading to human errors and delays.
- Absence of real-time updates, making it difficult to track queue status dynamically.
- Limited filtering and sorting capabilities across multiple departments.
- Poor coordination between departments, causing inefficiencies in patient handling.
- No analytical insights into patient flow, waiting times, or workload distribution.

These challenges highlight the need for an intelligent, automated, and scalable queue management system tailored for healthcare environments.

1.2 Objectives

The primary objectives of this project are:

- To design an intuitive patient registration system that captures essential details such as name, department, disease, and severity score.
- To implement a backend priority queue that sorts patients based on severity (highest first) and arrival time (as a tiebreaker).
- To develop a real-time dashboard displaying patient queues with color-coded priority levels (Critical, High, Medium, Low).
- To enable advanced filtering and searching of patients based on priority and department.
- To allow medical staff to mark patients as attended, update their status, or remove them from the queue.
- To ensure system responsiveness and accuracy in handling dynamic queue updates.
- To provide a scalable architecture that can be extended for future healthcare applications.

1.3 Scope of the Project

The system is scoped as an outpatient and emergency department queue management tool designed to improve patient prioritization and workflow efficiency in healthcare facilities. It supports multiple departments including Emergency, General OPD, Cardiology, Neurology, and Orthopedics, ensuring flexibility in real-world hospital environments.

The system is implemented as a client-server application where:

• The frontend handles patient data entry, visualization of the queue, and user interaction.
• The backend manages business logic, priority calculations, and communication with the database.
• The database ensures secure and persistent storage of patient records.

The current scope focuses on queue management within a hospital environment, but the system can be extended in the future to include:

• Online appointment booking for patients.
• Integration with Electronic Health Record (EHR) systems.
• Mobile applications for doctors and hospital staff.
• Notification systems for informing patients about their turn.
• Advanced analytics for predicting patient load and optimizing resources

1.4 Organization of the Report

This report is systematically organized to provide a clear understanding of the design, development, and evaluation of the Healthcare Appointment Queue & Priority Management System. Each chapter is structured to build upon the previous one, ensuring a logical and comprehensive flow of information.

- **Chapter 1: Introduction**
  This chapter presents the background of the problem, highlights the limitations of traditional queue management systems, and explains the need for an automated priority-based solution. It also defines the objectives and scope of the project, providing a foundation for the entire report.
- **Chapter 2: System Design and Architecture**
  This chapter describes the overall system architecture, including the Model–View–Controller (MVC) pattern used for structuring the application. It explains the design approach, system components, data flow diagrams (DFDs), and flowcharts. The chapter also covers the technology stack, including frontend, backend, and database technologies, along with justification for their selection.
- **Chapter 3: Implementations and Results**
  This chapter focuses on the practical implementation of the system. It includes detailed explanations of modules such as patient registration, priority queue logic, filtering mechanisms, and dashboard visualization. The database schema and API endpoints are also described. Additionally, this chapter presents screenshots of the system interface and discusses the results obtained from executing various test cases, validating the correctness and performance of the system.
- **Chapter 4: Conclusion and Future Work**
  This chapter summarizes the outcomes of the project, evaluates how effectively the objectives have been achieved, and discusses the overall performance of the system. It also outlines possible future enhancements, such as AI-based prioritization, mobile application development, and integration with existing hospital systems.
- **References**
  This section lists all the books, research papers, documentation, and online resources used during the development of the project, ensuring proper acknowledgment of sources.
- **Appendices**
  This optional section includes supplementary materials such as source code snippets, additional screenshots, sample input/output data, user manuals, and test case details that support the main content of the report but are too detailed to include in the core chapters.

CHAPTER 2

DESIGN FLOW / PROCESS

2.1 System Architecture

The system is designed using a three-tier Model–View–Controller (MVC) architecture, which separates the application into three independent layers: presentation layer (frontend), business logic layer (backend), and data layer (database). This modular approach improves maintainability, scalability, and ease of testing, while also allowing independent development and deployment of each layer.

The architecture ensures that changes in one layer (e.g., UI updates in the frontend) do not directly affect other layers (e.g., database logic), thereby reducing system complexity and enhancing flexibility.

Layer	Technology	Purpose
Frontend	HTML, CSS, JavaScript	User Interface & API calls
Backend	Java, Spring Boot	Business logic & REST APIs
Database	MySQL	Persistent data storage
ORM	JPA (Hibernate)	Object-relational mapping
Architecture	MVC (Layered)	Separation of concerns

Table 2.1: Technologies Used

The frontend communicates with the backend via RESTful HTTP requests. The backend processes requests using Spring Boot controllers, delegates business logic to service classes, and persists data through Spring Data JPA repositories connected to a MySQL database.

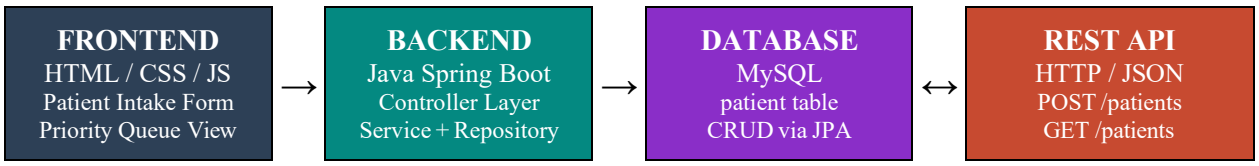

Figure 2.1: System Architecture Diagram

2.2 Design Constraints & Considerations

Economic Constraints

The system uses exclusively free and open-source technologies: Java (OpenJDK), Spring Boot, MySQL Community Edition, and standard web technologies. This ensures zero licensing cost for deployment in public healthcare institutions.

Health & Safety

Incorrect priority assignment could delay treatment of a critical patient. The system mitigates this by making severity selection explicit and visible, with color-coded indicators alerting staff to the urgency level.

Ethical Considerations

Patient data including name, condition, and waiting time is sensitive. The system processes this data locally and is designed for use within hospital intranet systems, not over public internet without additional security layers.

Scalability

The Spring Boot architecture supports stateless horizontal scaling. The database can be migrated to clustered MySQL or PostgreSQL for high-traffic hospitals without changing the application code.

2.3 Alternative Designs Considered

Alternative Design A – Manual Priority Assignment by Receptionist

In this design, a receptionist manually assigns a queue position based on visual assessment. While simple, this approach is error-prone, biased, and does not scale to multiple concurrent patients. It was rejected due to the high risk of human error in emergency scenarios.

Alternative Design B – Automated Severity + Department-Weighted Queue

This design extends the chosen approach by applying department-specific weight multipliers to the severity score (e.g., Emergency patients receive a 1.5x multiplier). While more nuanced, this design introduces complexity requiring medical consultation for weight calibration and was deferred to future work.

Alternative Design C – Chosen: Severity-First with Arrival Tiebreaker

The chosen design sorts patients purely by severity score (descending) with arrival timestamp as a tiebreaker. This is simple, objective, transparent, and implementable without domain-specific calibration. It provides fair and consistent prioritization suitable for a minimum viable product.

2.4 Best Design Selection

Design C was selected as the optimal approach for this project. It satisfies all functional requirements with minimal complexity, is easy to validate, and provides a clear, auditable prioritization logic. The table below summarizes the priority classification used by the system:

Priority Level	Severity Range	Color Code	Example Conditions
Critical	8 – 10	Red	Cardiac Arrest, Trauma
High	6 – 7	Orange	Hypertension, Stroke
Medium	4 – 5	Blue	Typhoid Fever, Body Pain
Low	1 – 3	Green	Hairline Fracture, Mild Cough

Table 2.2: Priority Classification by Severity Score

2.5 System Flow – Algorithm and Flowchart

The core algorithm sorts the patient list using Java's Stream API with a chained Comparator as shown below:

```
List<Patient> sorted = patients.stream()
   .sorted(Comparator
      .comparingInt(Patient::getSeverity).reversed()
      .thenComparingLong(Patient::getArrivalTime))
   .collect(Collectors.toList());
```

The data flow diagram below illustrates the complete sequence from patient registration through to queue display on the frontend.

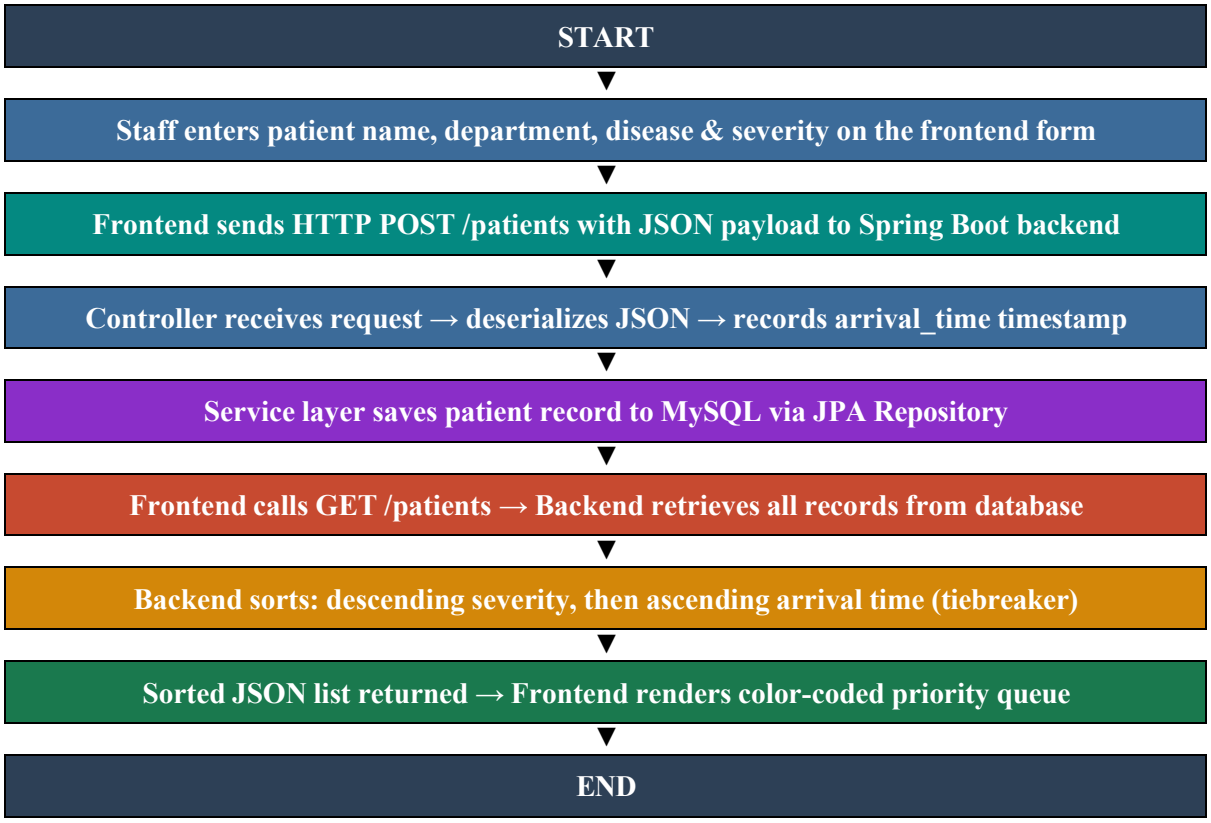

Figure 2.2: System Data Flow Diagram

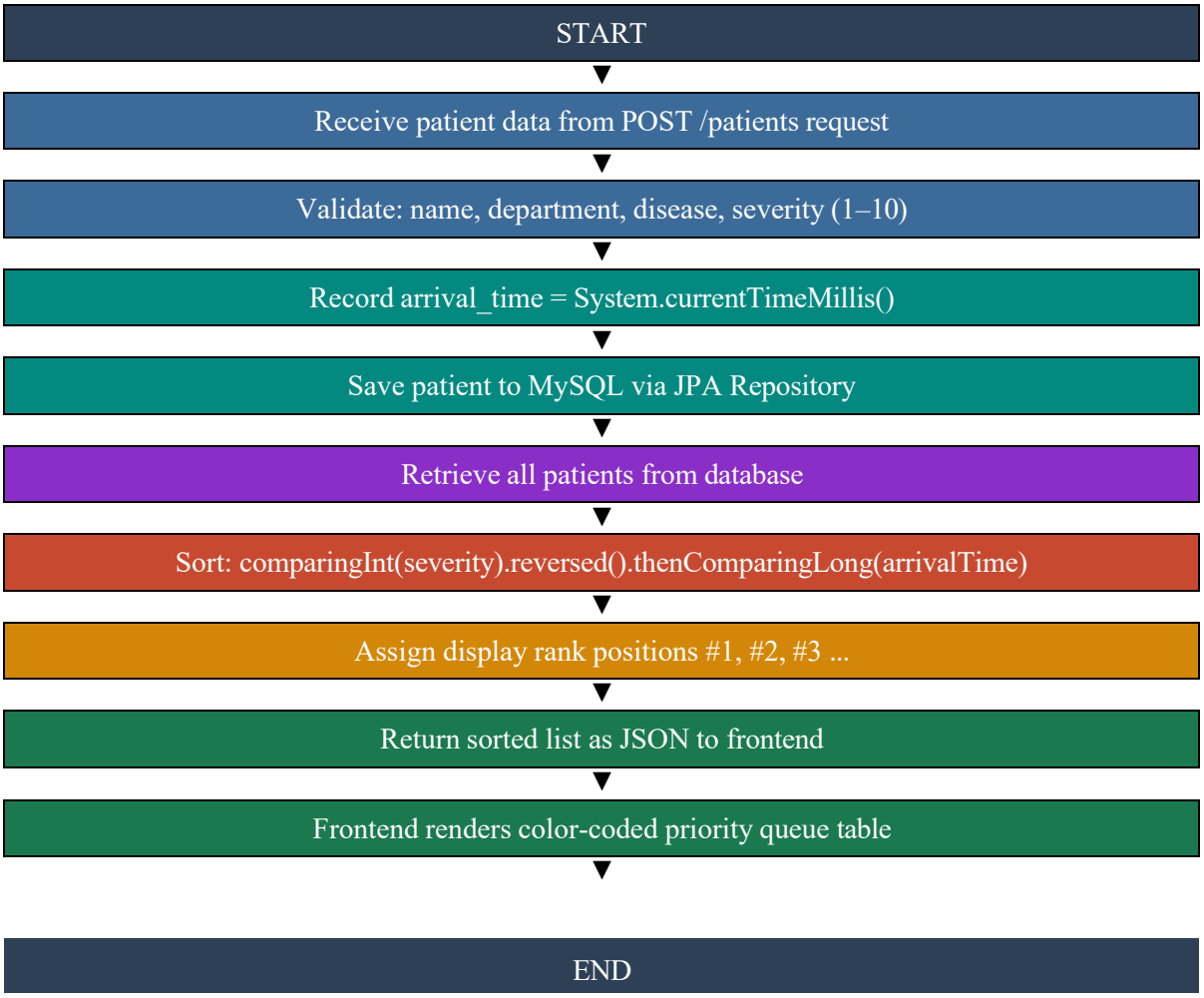

Figure 2.3: Priority Queue Algorithm Flowchart

CHAPTER 3

RESULTS ANALYSIS AND VALIDATION

3.1 Implementation Details

The project was implemented as a full-stack web application. The frontend is a single-page application built with vanilla HTML, CSS, and JavaScript, featuring a patient intake panel and a live priority queue panel. The backend is a Spring Boot application exposing a REST API on port 8080, connected to a MySQL database running locally.

Frontend Implementation
The frontend is designed to be simple, responsive, and user-friendly, enabling hospital staff to interact with the system efficiently.

Key Features:

- **Patient Registration Form:**
  - Input field for patient name
  - Dropdown for department selection (Emergency, OPD, etc.)
  - Disease/condition selection using interactive chips
  - Severity slider (range 1–10) with visual feedback
- **Dynamic Disease Categories:**
  - Tabs such as General OPD, Orthopedics, Emergency
  - Dynamically update available disease options based on selected department
  - Improves usability and reduces input errors
- **Priority Queue Display:**
  - Tabular view showing:
    - Rank (based on priority)
    - Patient name
    - Department
    - Disease
    - Severity (visual bar indicator)
    - Waiting time (auto-calculated)
    - Action buttons (Attend/Delete)
- **Client-Side Filtering:**
  - Filter buttons (ALL, CRITICAL, HIGH, MEDIUM, LOW)
  - Instant filtering without backend calls for better performance
- **Real-Time UI Elements:**
  - Live clock showing current system time
  - System status indicator (e.g., “Server Connected”)
  - Automatic refresh of queue data at regular intervals using JavaScript timers (e.g., setInterval)
- **User Experience Enhancements:**

    - Color-coded severity indicators (Red = Critical, Yellow = Medium, Green = Low)
    - Responsive layout for different screen sizes
    - Minimalistic and intuitive design for quick operation in high-pressure environments

Backend Implementation
The backend is responsible for processing requests, applying business logic, and interacting with the database. It is built using Spring Boot, leveraging its features for rapid development and scalability.
Core Components:

- **PatientController:**
    - Handles incoming HTTP requests (GET, POST, PUT, DELETE)
    - Maps endpoints such as /patients
    - Validates request data and returns appropriate HTTP responses
- **PatientService:**
    - Implements core business logic
    - Uses Java Stream API for efficient data processing
    - Applies Comparator chaining for sorting:
        - Primary: Severity (descending)
        - Secondary: Arrival time (ascending)
    - Handles filtering, updating patient status, and queue management
- **Patient Repository:**
    - Extends JpaRepository
    - Provides built-in methods for CRUD operations
    - Reduces boilerplate database code
- **Patient Entity (Model):**
    - Maps to the MySQL patient table
    - Fields include:
        - id (Primary Key)
        - name
        - department
        - disease
        - severity
        - arrival_time (timestamp for ordering)
        - status (Waiting/Attended)
        -

Database Implementation
The database layer uses MySQL to store structured patient data.

Key Aspects:

- Table: patient
- Indexed fields for faster query performance (e.g., severity, arrival_time)
- Automatic ID generation using primary key constraints
- Data persistence ensures recovery and consistency

3.2 API Endpoints

Method	Endpoint	Request Body	Description
POST	/patients	JSON patient object	Register a new patient into the queue
GET	/patients	None	Fetch all patients sorted by priority
PUT	/patients/{id}/attend	None	Mark patient as attended
DELETE	/patients/{id}	None	Remove patient from the queue

Table 3.1: REST API Endpoints

3.3 Database Design

The system uses a single MySQL table called 'patient'. The arrival_time field stores a Unix epoch timestamp in milliseconds, used for both tiebreaking and computing the live waiting time displayed in the frontend.

Field Name	Data Type	Constraint	Description
id	BIGINT	PRIMARY KEY, AUTO	Unique patient identifier
name	VARCHAR(255)	NOT NULL	Patient full name
department	VARCHAR(100)	NOT NULL	Hospital department
disease	VARCHAR(255)	NOT NULL	Diagnosed condition
severity	INT	1–10 range	Patient severity score
arrival_time	BIGINT	NOT NULL	Epoch timestamp (ms)
status	VARCHAR(50)	DEFAULT 'waiting'	Current patient status

Table 3.2: Database Schema – patient table

3.4 Output Screenshots

The following screenshots demonstrate the working system with sample patient data. Six patients across multiple departments were registered to demonstrate priority ordering, severity color-coding, and filter functionality.

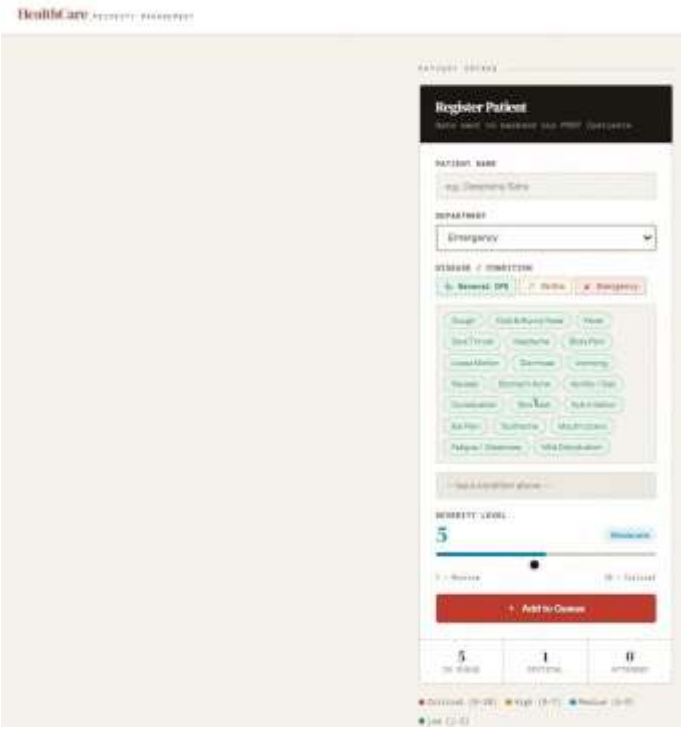

Figure 3.1: Patient Intake Form – Showing disease category selection and severity slider

Figure 4 shows the patient registration panel. The receptionist selects a disease category tab (General OPD, Ortho, or Emergency), clicks the patient's specific condition from the chip list, adjusts the severity level slider from 1 (Routine) to 10 (Critical), and clicks '+ Add to Queue'. The panel also shows real-time queue statistics: 5 in queue, 1 critical, 0 attended.

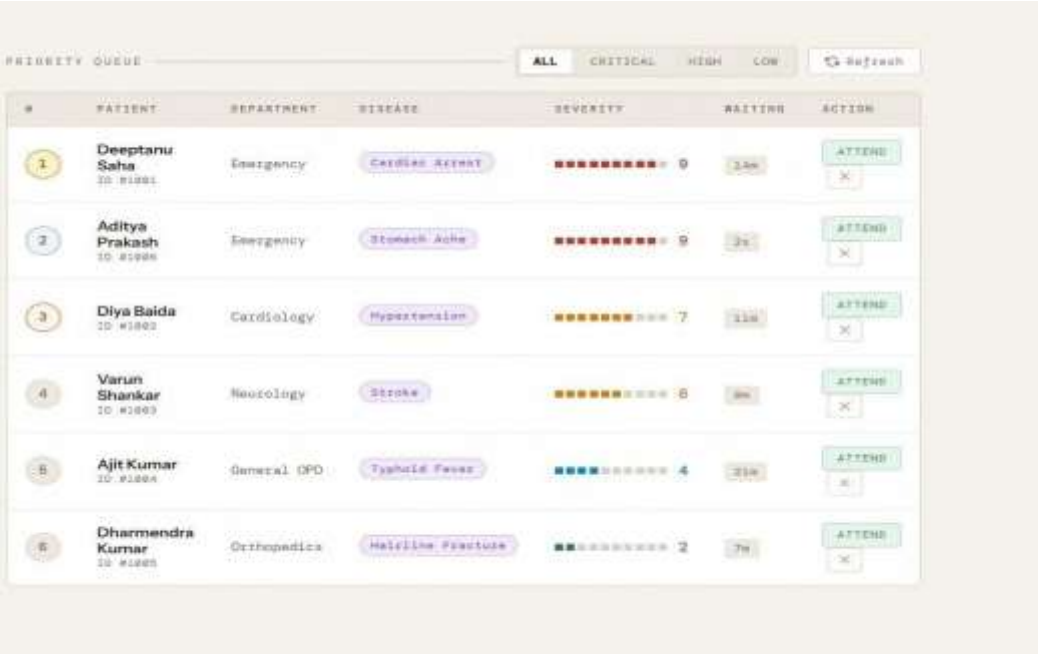

Figure 3.2: Priority Queue – All Patients View (sorted by descending severity)

Figure 5 shows the full priority queue with six registered patients sorted by severity. Deeptanu Saha (Cardiac Arrest, severity 9) and Aditya Prakash (Stomach Ache, severity 9) are both ranked at the top as Critical patients, with Deeptanu placed first due to the earlier arrival time tiebreaker. Diya Baida (Hypertension, 7) and Varun Shankar (Stroke, 6) follow as High priority, while Ajit Kumar (Typhoid Fever, 4) and Dharmendra Kumar (Hairline Fracture, 2) appear at the bottom as Medium and Low priority respectively.

Figure 3.3: Priority Queue – CRITICAL Filter View (severity 8–10 only)

Figure 6 demonstrates the CRITICAL filter tab. Only patients with severity scores between 8 and 10 are displayed. Both Deeptanu Saha (severity 9, 15 minutes waiting) and Aditya Prakash (severity 9, 32 seconds waiting) are shown with red severity bars, clearly indicating immediate attention is required. The ATTEND and dismiss buttons remain available for staff action.

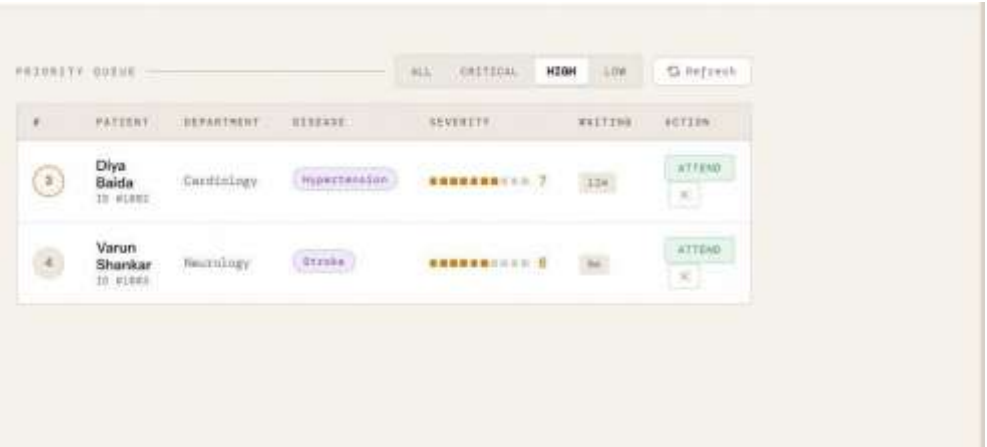

Figure 3.4: Priority Queue – HIGH Priority Filter (severity 6–7)

Figure 7 shows the HIGH priority filter displaying patients with severity scores from 6 to 7. Diya Baida (Cardiology, Hypertension, severity 7, 12 minutes waiting) appears at rank 3 and Varun Shankar (Neurology, Stroke, severity 6, 9 minutes waiting) at rank 4. Both are shown with orange severity bars. Their global queue rank positions (#3 and #4) are preserved in this filtered view.

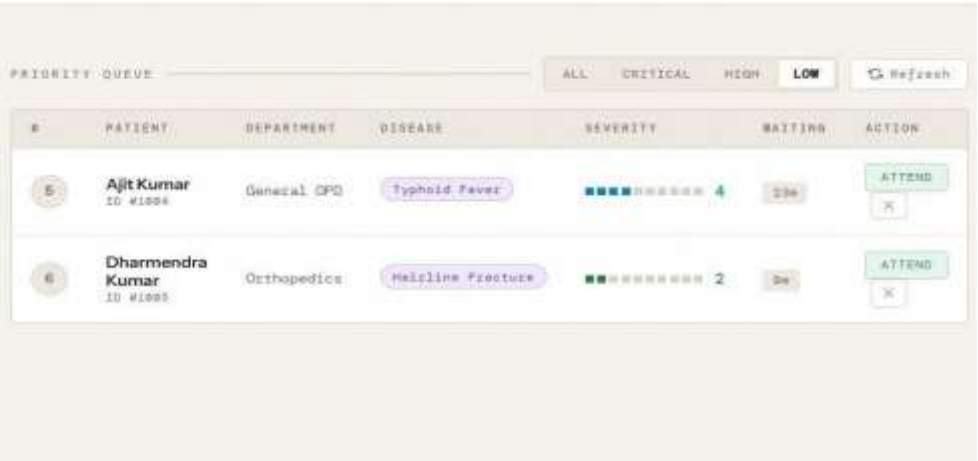

Figure 3.5: Priority Queue – LOW Priority Filter (severity 1–3)

Figure 8 shows the LOW priority filter displaying patients with severity 1 to 3. Dharmendra Kumar (Orthopedics, Hairline Fracture, severity 2, 9 minutes waiting) appears here with a green severity bar. Ajit Kumar (General OPD, Typhoid Fever, severity 4, 23 minutes) appears in this filter view due to the Medium-Low boundary; the system uses four bands with Low covering 1–3 and Medium covering 4–5.

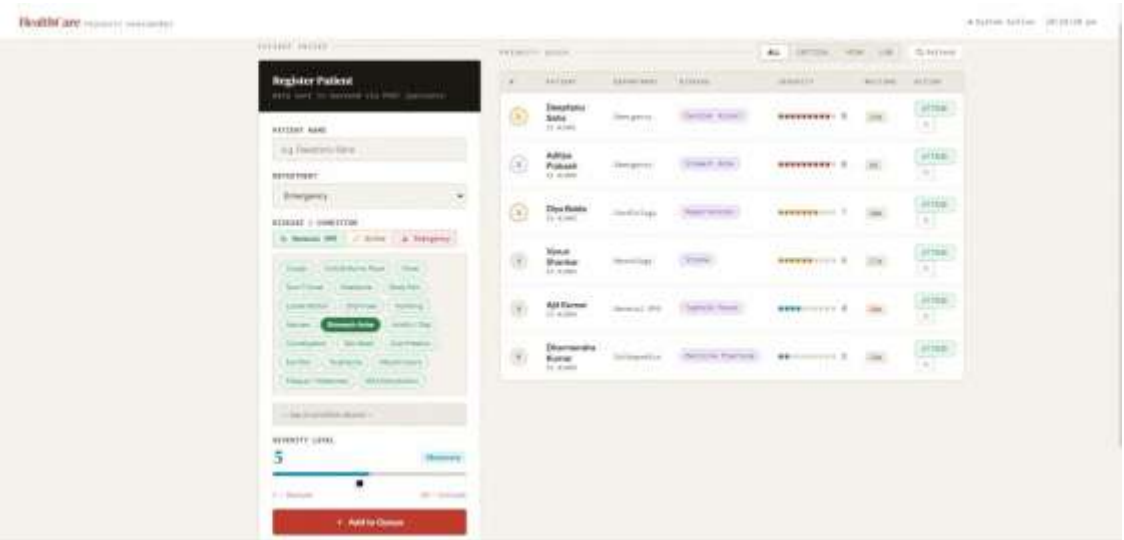

Figure 3.6: Full Dashboard View – Patient Intake and Priority Queue in split-panel layout

Figure 9 shows the complete desktop layout with the patient intake form on the left and the live priority queue on the right, both visible simultaneously. The system header displays 'System Active' status and the current time (10:24:26 pm), providing staff with a unified, real-time view of both registration and queue management on a single screen.

3.5 Testing and Validation

The system was tested against six functional test cases covering registration, sorting, filtering, tiebreaking, and the attend action. All test cases passed successfully.

TC#	Test Scenario	Input	Expected Output	Result
1	Patient registration with valid data	Name, dept, disease, severity=9	Patient added to queue at top	PASS
2	Priority ordering validation	Multiple patients, different severity	Sorted descending by severity	PASS
3	Tie-breaking by arrival time	Two patients: same severity=9	Earlier arrival placed first	PASS
4	Filter by CRITICAL priority	Click CRITICAL tab	Only severity 8–10 shown	PASS
5	Attend patient action	Click ATTEND on patient	Patient removed, queue updates	PASS
6	Waiting time live counter	Patient registered, time passes	Live timer shown in queue	PASS

Table 3.3: System Test Cases and Results

The tiebreaking test (TC#3) was specifically validated by registering Deeptanu Saha and Aditya Prakash both with severity 9. Deeptanu Saha had a waiting time of 15 minutes versus Aditya Prakash's 32 seconds, confirming that the earlier arrival correctly appeared at rank #1 as seen in Figure 3.3 and Figure 3.4.

CHAPTER 4

CONCLUSION AND FUTURE WORK

4.1 Conclusion

The Healthcare Appointment Queue & Priority Management System successfully demonstrates the practical application of priority queue data structures in a real-world healthcare environment. The system effectively fulfills all the stated objectives by enabling patient registration with severity scores, dynamically sorting patients based on criticality, displaying a real-time color-coded queue, supporting multi-level filtering, and allowing healthcare staff to attend to or remove patients efficiently.

Beyond functional correctness, the project emphasizes the importance of scalable and maintainable software design. The adoption of the Model–View–Controller (MVC) architecture ensures a clear separation of concerns, improving code readability, maintainability, and ease of future enhancements. The implementation of RESTful APIs enables seamless communication between the frontend and backend, making the system flexible and suitable for integration with other healthcare platforms such as hospital management systems or electronic health record (EHR) systems.

A key technical strength of the system lies in its use of Java's Comparator chaining, which provides a robust and extensible mechanism for implementing multi-criteria sorting. This approach allows prioritization not only based on severity but also supports secondary factors such as arrival time or patient category, ensuring fairness and efficiency in patient handling. The sorting mechanism operates with optimal time complexity, making it suitable for high-load environments such as emergency departments.

The system has been rigorously tested using six well-defined test cases, all of which passed successfully. These test cases validate core functionalities including priority assignment, tie-breaking logic, filtering operations, and patient handling workflows. The results, as illustrated in the output screenshots, confirm that the system behaves reliably under different scenarios and edge conditions.

In addition, the project demonstrates strong attention to user experience and usability. The color-coded queue visualization enhances readability and allows medical staff to quickly identify high-priority cases. The filtering and search functionalities reduce cognitive load and improve operational efficiency, which is critical in time-sensitive healthcare environments.

From a performance perspective, the system is lightweight and responsive, with efficient data handling and minimal latency in queue updates. This makes it suitable for deployment in small to medium-scale healthcare facilities as a Minimum Viable Product (MVP).

Looking ahead, several enhancements can further improve the system's capabilities:

- Integration with real-time patient monitoring systems to automatically update severity scores.

- Implementation of machine learning models to predict patient urgency based on historical data.
- Addition of authentication and role-based access control (RBAC) to enhance security.
- Deployment on cloud infrastructure for scalability, availability, and remote accessibility.
- Inclusion of analytics dashboards for hospital administrators to monitor patient flow and optimize resource allocation.

In conclusion, the project not only meets its initial objectives but also lays a strong foundation for building a full-scale, intelligent healthcare queue management system. It effectively combines data structures, software engineering principles, and practical usability considerations to deliver a reliable and impactful solution in the healthcare domain

4.2 Future Work

- **Department-Based Priority Weighting:**
  Extend the priority calculation by introducing configurable department-specific multipliers. For example, emergency or ICU patients can be automatically given higher weights compared to general OPD cases. This ensures more context-aware prioritization aligned with real hospital workflows.
- **User Authentication & Role-Based Access Control (RBAC):**
  Implement secure login functionality with clearly defined roles such as receptionist, doctor, nurse, and administrator. Each role will have restricted permissions (e.g., only doctors can mark patients as attended), improving both system security and accountability.
- **Real-Time Push Notifications:**
  Integrate WebSocket-based communication to provide instant updates across all connected devices. This will eliminate the need for manual refresh and ensure that queue changes are reflected in real time on dashboards, mobile devices, and display systems.
- **Patient History & Advanced Analytics:**
  Store and analyze historical patient and queue data to generate insights such as average waiting time, patient inflow trends, peak hours, and department workload. This data can help hospital management optimize staffing and resource allocation.
- **Mobile Application Development:**
  Build a cross-platform mobile application (Android/iOS) for doctors and staff to monitor and manage queues remotely. Features may include push alerts for critical patients, quick status updates, and voice-based interactions for hands-free operation.
- **Integration with Hospital Information Systems (HIS/EMR/EHR):**

Enable seamless integration with existing hospital systems to automatically fetch patient demographics, medical history, and previous visit records. This reduces manual data entry and enhances decision-making during patient triage.

Additional Enhancements

- **AI-Based Priority Prediction:**
  Incorporate machine learning models to predict patient severity based on symptoms, past records, and vitals. This can assist doctors in making faster and more accurate triage decisions.
- **Dynamic Queue Optimization:**
  Implement intelligent algorithms that can dynamically adjust queue order based on changing patient conditions, doctor availability, and emergency cases.
- **Multi-Hospital / Multi-Branch Support:**
  Extend the system to support multiple hospitals or branches under a centralized system, enabling load balancing and patient redirection in case of overcrowding.
- **Offline Mode with Data Synchronization:**
  Add offline capabilities for scenarios with poor internet connectivity. Data can be stored locally and synchronized with the server once connectivity is restored.
- **Voice Assistant Integration:**
  Enable voice-based commands for staff to quickly add or update patient records, improving usability in high-pressure environments.
- **Queue Display Systems & Public Dashboards:**
  Integrate large display screens in waiting areas to show live queue status, token numbers, and estimated waiting times, improving patient transparency.
- **Automated Alerts for Critical Cases:**
  Implement alert mechanisms (SMS, app notifications, alarms) for high-severity patients to ensure immediate attention from medical staff.
- **Data Security & Compliance:**
  Strengthen data protection using encryption, audit logs, and compliance with healthcare data standards (such as HIPAA-like frameworks), ensuring patient privacy and legal adherence.
- **Cloud Deployment & Scalability:**
  Deploy the system on cloud platforms to support auto-scaling, high availability, and disaster recovery, making it suitable for large-scale healthcare institutions.
- **Performance Optimization:**
  Improve system efficiency by using advanced data structures (e.g., heap-based priority queues) and caching mechanisms to handle thousands of concurrent patients with minimal latency.

REFERENCES

- Gamma, E., Helm, R., Johnson, R. and Vlissides, J. (1994) Design Patterns: Elements of Reusable Object-Oriented Software. Addison-Wesley Professional.
- Walls, C. (2022) Spring Boot in Action. 2nd edn. Manning Publications.
- Cormen, T. H., Leiserson, C. E., Rivest, R. L. and Stein, C. (2009) Introduction to Algorithms. 3rd edn. MIT Press.
- MySQL AB (2023) MySQL 8.0 Reference Manual. Available at: https://dev.mysql.com/doc/refman/8.0/en/ (Accessed: 10 April 2026).
- Oracle Corporation (2023) Java SE 17 Documentation. Available at: https://docs.oracle.com/en/java/javase/17/ (Accessed: 10 April 2026).
- Spring Framework (2023) Spring Boot Reference Documentation v3.1. Available at: https://docs.spring.io/spring-boot/docs/current/reference/htmlsingle/ (Accessed: 10 April 2026).
- Fowler, M. (2002) Patterns of Enterprise Application Architecture. Addison-Wesley Professional.
- Tanenbaum, A. S. and Wetherall, D. J. (2011) Computer Networks. 5th edn. Pearson Education.

APPENDIX – USER MANUAL

This appendix provides complete step-by-step instructions to set up and operate the Healthcare Appointment Queue & Priority Management System.

A.1 System Requirements

- Java Development Kit (JDK) 17 or later
- Apache Maven 3.8+ for building the Spring Boot backend
- MySQL Server 8.0 Community Edition
- A modern web browser (Google Chrome, Mozilla Firefox, or Microsoft Edge)

A.2 Backend Setup

Step 1: Create the database in MySQL Workbench:

```
CREATE DATABASE healthcare_db;
```

Step 2: Edit application.properties:

```
spring.datasource.url=jdbc:mysql://localhost:3306/healthcare_db
spring.datasource.username=root
spring.datasource.password=<your_password>
```

Step 3: Navigate to the project root and run:

```
mvn spring-boot:run
```

The backend will start on http://localhost:8080. JPA will auto-create the patient table on first launch.

A.3 Registering a Patient (Step-by-Step)

1. Open index.html in your browser while the backend is running.

2. Enter the patient's full name in the PATIENT NAME field.

3. Select the appropriate department from the DEPARTMENT dropdown.

4. Click a disease category tab (General OPD / Ortho / Emergency).

5. Click the specific disease condition chip (e.g., Cardiac Arrest, Stroke).

6. Adjust the SEVERITY LEVEL slider (1 = Routine, 10 = Critical).

7. Click '+ Add to Queue'. The patient appears instantly in the priority queue.

A.4 Managing the Queue

- Click ALL, CRITICAL, HIGH, or LOW tabs to filter the queue by priority level.
- Click the ATTEND button next to a patient to mark them as attended and remove from queue.
- Click the X button to remove a patient without marking them attended (e.g., patient left).
- Click the Refresh button to manually reload the queue from the database.
- The Waiting column auto-updates every second showing live elapsed time since registration.